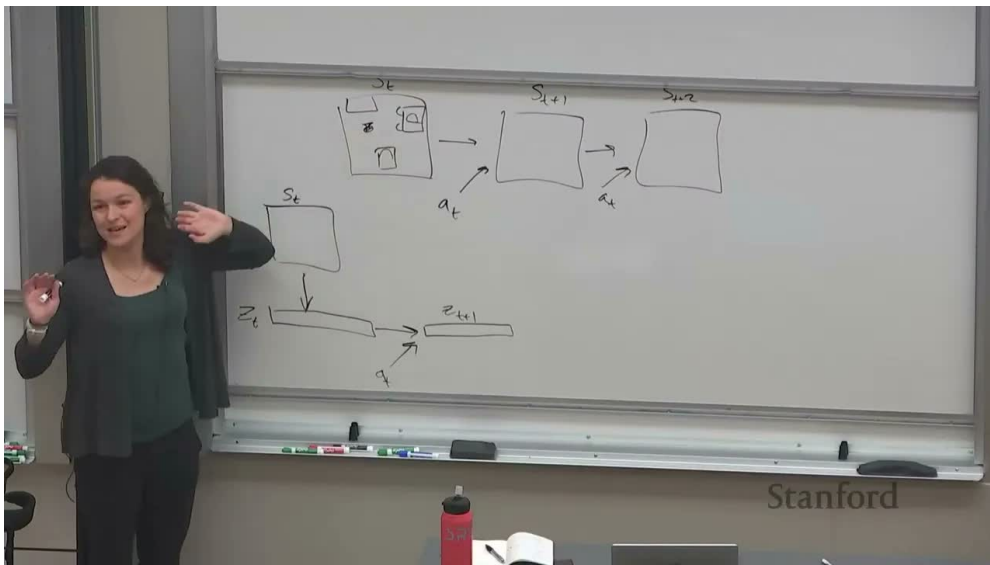


课程笔记

CS224R Lecture 11: Model-Based RL

Codex

2026-04-13



视频作者/频道: Stanford CS224R

发布日期: 2025-05-07

视频时长: 约 01:13:20

视频链接: <https://www.youtube.com/watch?v=PvqyGn0irgA>

目录

1	Model-Based RL 的核心问题：能不能先学一个世界模型	2
1.1	一个最基础的 learned simulator pipeline	2
1.2	本章小结	3
2	第一条路：在模型里直接规划	3
2.1	把动作序列当作优化变量	3
2.2	梯度法与采样法的差异	4
2.3	本章小结	5
3	为什么直接规划会失败，以及 MPC 如何补救	5
3.1	数据分布失配会让模型在关键区域失真	5
3.2	open-loop 为什么脆弱	5
3.3	MPC: planning 也要闭环	6
3.4	本章小结	6
4	第二条路：不用模型直接决策，而是用模型扩充数据	7
4.1	为什么 imagined rollouts 更实用	7
4.2	什么时候 model-based 值得做	7
4.3	本章小结	8
5	案例：灵巧手操控为什么适合做 model-based RL	8
5.1	任务定义与为什么它是好案例	8
5.2	为什么 ensemble 和改造版 CEM 有帮助	9
5.3	真实世界部署说明了什么	9
5.4	本章小结	10
6	总结与延伸	10

1 Model-Based RL 的核心问题：能不能先学一个世界模型

这节课把强化学习分成一个非常清楚的谱系：offline / online、imitation / RL、model-free / model-based。Model-based RL 的核心出发点并不复杂，就是先学一个能预测未来的模型，再利用这个模型帮助决策。讲者把它称为“learned simulator”，这比把它仅仅理解成下一状态预测器更准确，因为它承担的是环境近似器的角色。

Can we learn a “simulator”?

i.e. predictive model of $\mathbf{s}_{t+1}$ given $\mathbf{s}_t, \mathbf{a}_t$

Robotics & physical systems

modeling physics of (directly observed) physical system

video prediction, conditioned on actions



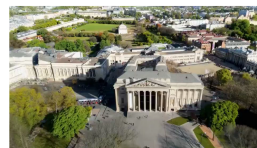
Vevo 2

Finance: stock market predictor

Games: rules of the game

May need to model other players —

Sometimes considered part of dynamics, sometimes modeled separately (multi-agent RL)



Sora

May not need to learn a model! (e.g. chess)

6

图 1: learned simulator 的适用场景：物理系统、视频预测、金融市场、游戏规则等，都可以视为状态转移规律的建模问题。

讲者特别强调，“模型”并不一定意味着高精度物理方程。对于机器人和驾驶，模型可能近似的是显式物理世界；对于难以观测底层变量的场景，模型更像 action-conditioned video prediction；在游戏中，规则本身就是环境模型；而在多智能体设置里，其他玩家有时也被当作动力学的一部分。

因此，model-based RL 的第一层判断不是“模型有没有用”，而是“这个领域有没有希望学出足够有用的模型”。如果答案是否定的，那么后面所有 planning 与 imagined rollouts 都会被系统性误导。

1.1 一个最基础的 learned simulator pipeline

课程给出的第一版 pipeline 非常朴素：

1. 用某个基础策略采集数据集 $\mathcal{D} = \{(s, a, s')_i\}$;
2. 训练动力学模型 $f_\phi(s, a)$ 去预测 s' ;
3. 在学到的模型内部做动作优化或规划;
4. 将规划后的动作拿回真实环境执行。

最直接的训练目标就是最小化一步预测误差：

$$\min_{\phi} \sum_i \|f_\phi(s_i, a_i) - s'_i\|_2^2.$$

这一定义本身已经暴露了 model-based RL 的两个关键前提：一是你必须覆盖足够广的数据，二是你希望一步预测误差能够外推到多步决策。

本讲主线

Lecture 11 的核心不是背算法名，而是建立一个判断框架：学到的模型如何被使用？为什么 planning 会失败？什么时候该 replan？什么时候应该直接用模型生成额外数据，再交给 model-free RL 来更新策略？

1.2 本章小结

开场部分建立了 model-based RL 的最基本视角：先学模型，再用模型。真正困难的地方不在于写下 $f_\phi(s, a)$ ，而在于这个模型到底要被怎样调用，以及它的误差会在决策中如何被放大。

2 第一条路：在模型里直接规划

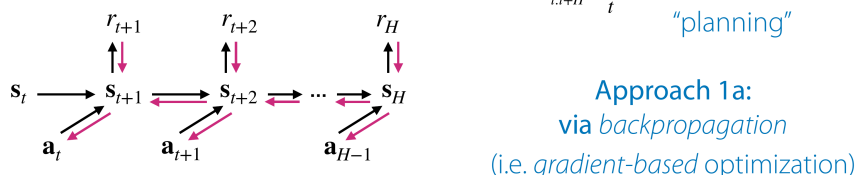
2.1 把动作序列当作优化变量

如果模型 f_ϕ 已经能近似预测未来，那么一个自然做法是直接对未来动作序列做优化。对于规划视角，目标可以写成

$$\max_{a_{t:t+H-1}} \sum_{h=0}^{H-1} r(s_{t+h}, a_{t+h}) \quad \text{s.t.} \quad s_{t+h+1} = f_\phi(s_{t+h}, a_{t+h}).$$

这里真正被优化的不是网络参数，而是未来的动作序列 $a_t, \dots, a_{t+H-1}$ 。这和常规神经网络训练很不一样：我们把模型固定下来，然后在模型内部搜索一串最优控制。

Approach 1: Optimize over actions using model $\max_{a_{t:t+H}} \sum_t r(s_t, a_t)$



Algorithm:

1. Run some policy (e.g. random policy) to collect data $\mathcal{D} = \{(s, \mathbf{a}, s')\}$
2. Learn model $f_\phi(s, \mathbf{a})$ to minimize $\sum_i \|f_\phi(s_i, \mathbf{a}_i) - s'_i\|^2$
3. Backpropagate through $f_\phi(s, \mathbf{a})$ to choose actions

12

图 2: 通过 learned model 直接对动作序列做规划：先学 $f_\phi(s, a)$ ，再在模型里搜索高回报动作。

讲者把这类做法称为 planning。它的优势是：不需要先学出一个显式 policy 网络，只要模型和奖励足够可信，就能在当前状态上“向前想”。但代价也很直接，推理时会很重，因为每到一个状态都得重新做优化。

2.2 梯度法与采样法的差异

课程把规划里的动作优化分成两大类。第一类是**梯度法**，即 reward 对动作可微时，通过反向传播直接更新动作序列。第二类是**采样法**，即不要求可微，只需要能评估某串动作的累计回报。

Aside: Gradient-based vs. sampling-based optimization

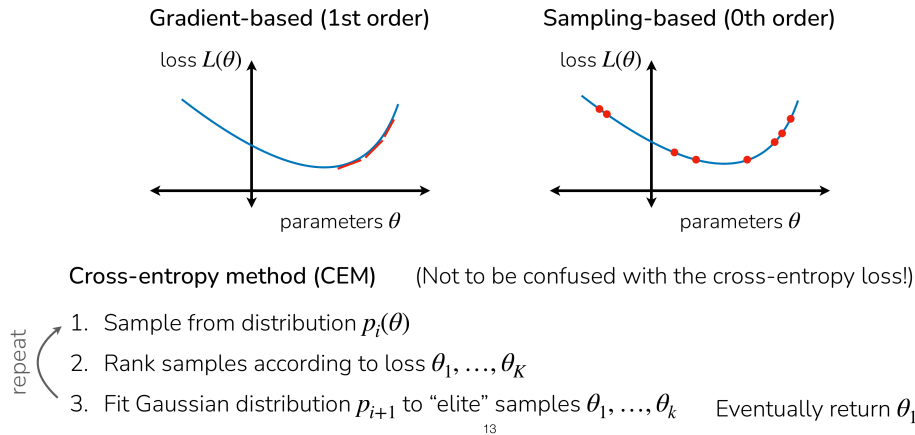


图 3: 梯度优化与采样优化的对比：前者依赖可微结构，后者更黑盒，但通常更稳健。

讲者对采样法的说明尤其重要。最简单的是 random shooting：随机采样很多候选动作序列，逐个 rollout，选回报最高的一组。更高级的是 cross-entropy method, CEM：保留 elite sequences，再拟合一个新的采样分布迭代更新。CEM 的优点是简单、易并行、对不可微模型友好；缺点是动作维度和规划 horizon 一大，就会迅速变贵。

Sampling-Based Optimization

Denote $\mathbf{A} := \mathbf{a}_t, \dots, \mathbf{a}_{t+H}$

Version 1: Guess & check

"random shooting"

- a. Sample many $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g. uniform)
- b. Choose $\mathbf{A}_i$ based on $\arg \max_i \sum_{t'=t}^{t+H} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$ Can we improve this distribution?

Version 2: Cross-entropy method

- a. Sample many $\mathbf{A}_1, \dots, \mathbf{A}_N$ from $p(\mathbf{A})$
 - b. Evaluate $J(\mathbf{A}_i) = \sum_{t'=t}^{t+H} r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$
 - c. Pick the elites $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$ with the largest $J(\mathbf{A}_i)$, where $M < N$
 - d. Refit $p(\mathbf{A})$ to the elites $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$
- + fast, if parallelized
+ simple
- doesn't scale to high dimensions
(including both H and $|\mathbf{a}|$)

16

图 4: CEM 式采样规划：采样候选动作序列、评估回报、保留 elite、再拟合采样分布。

2.3 本章小结

如果模型可用，那么 planning 是最直接的 model-based RL 方案。它的思想简单、目标清晰，但它把困难从“学策略”转移到了“每一步都要在线搜索”。因此，planning 的实际可行性高度依赖模型是否可靠、动作空间是否可控，以及测试时预算是否允许。

3 为什么直接规划会失败，以及 MPC 如何补救

3.1 数据分布失配会让模型在关键区域失真

讲者用一个“向右走会上高地，但继续走会掉下悬崖”的例子解释 model-based planning 的经典失败模式。训练数据来自某个基础策略 π_0 ，而最终规划出的策略 π_f 会访问完全不同的状态区域，于是出现

$$p_{\pi_0}(s) \neq p_{\pi_f}(s).$$

模型在训练数据覆盖的区域也许很准，但在新访问到的状态上会产生系统性误差。规划器又会主动寻找高回报区域，于是极容易被这些未见区域里的虚假高值吸引。

How can this approach fail?



1. Run some policy (e.g. random policy) to collect data $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')\}_i$
2. Learn model $f_\theta(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f_\theta(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. Iteratively sample action sequences, run through model $f_\theta(\mathbf{s}, \mathbf{a})$ to choose actions

Data distribution mismatch

$$p_{\pi_0}(\mathbf{s}) \neq p_{\pi_f}(\mathbf{s})$$

Going right means that we can go higher!

Thought Exercise: How might you alleviate this issue?

18

图 5: direct planning 的典型失败：模型只在旧策略数据分布上准确，到了新状态区域就会犯系统性错误。

这也是为什么 model-based RL 不能只离线学一次模型就完事。讲者在口头解释里明确说到：当你执行规划动作后，应把新数据加入数据集、重新拟合模型，再继续迭代。只有这样，模型才会逐渐覆盖最终策略真正访问的区域。

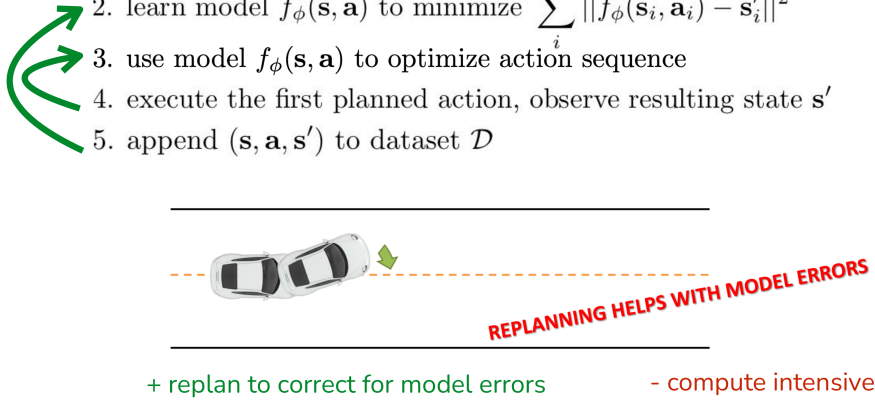
3.2 open-loop 为什么脆弱

另一类失败并不来自训练分布，而来自执行方式。若你一次规划出整串动作 $a_{t:t+H-1}$ ，然后不再看环境反馈、直接全都执行完，这就是 open-loop control。它的问题是：任何一个中途偏差，无论来自环境扰动还是模型误差，都会让后续动作建立在错误假设上。

Approach 2: Plan & replan using model

model-predictive control (MPC)

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn model $f_\phi(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. use model $f_\phi(\mathbf{s}, \mathbf{a})$ to optimize action sequence
4. execute the first planned action, observe resulting state $\mathbf{s}'$
5. append $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to dataset $\mathcal{D}$



22

图 6: MPC 的核心思想是 “plan & replan”: 每次只执行第一个动作, 再根据真实新状态重新规划。

讲者在课堂讨论里给了一个非常工程化的解释: 如果世界变化很快, 或者模型没法精确预测未来, 那么 open-loop 的长动作序列其实相当危险, 因为你根本没有机会根据真实反馈修正自己。

3.3 MPC: planning 也要闭环

为了解决 open-loop 的脆弱性, 课程引入 model-predictive control, MPC。它仍然规划一个有限 horizon 的动作序列, 但只执行第一个动作; 得到真实下一状态后, 再重新规划剩余序列。这相当于把 planning 放进一个闭环反馈系统里。

MPC 的优点很明确:

- 可以根据真实新状态修正偏差;
- 对局部模型误差更鲁棒;
- 能把长程任务分解成一系列短程滚动规划问题。

它的代价同样明显: 每一步都要重新规划, 所以测试时计算开销很大。讲者还特别指出, planning 通常只适合短 horizon 问题, 因为一方面长 horizon 的搜索太贵, 另一方面模型反复 rollout 的误差会累积。

3.4 本章小结

这一部分给出了 planning 能不能落地的判断标准。只要出现数据分布失配、环境变化、模型误差累积, 单次 open-loop 规划都会变得不可靠。MPC 的意义不在于把 planning 变简单, 而在于把它从脆弱的离线计划, 变成一个能利用环境反馈持续纠错的闭环过程。

4 第二条路：不用模型直接决策，而是用模型扩充数据

4.1 为什么 imagined rollouts 更实用

讲者随后转向另一条思路：既然 planning 测试时代价大、长 horizon 下也不稳，那不如用 learned model 生成额外训练数据，再交给熟悉的 model-free RL 更新 policy。这就是 model-based policy optimization 的核心直觉。

关键不是从初始状态生成很长的虚拟轨迹，而是从真实数据中的很多状态出发，生成短的 **synthetic rollouts**。这样做能在真实数据附近增加局部覆盖，同时避免模型在长 horizon 下越滚越偏。

Model-based policy optimization

Key idea: augment data with model-simulated roll-outs.

Full algorithm

- 
1. Collect data using current policy π_ϕ , add to D_{env}
 2. Update model $p_\theta(s'|s, a)$ using D_{env}
 3. Collect synthetic roll-outs using π_ϕ in model p_θ from states in D_{env} ; add to D_{model}
 4. Update policy π (and critic Q) using D_{model}

Notes: - compatible with variety of model-free RL methods (step 4)
- could additionally use D_{env} in policy update

28

图 7: model-based policy optimization 的全算法：真实环境数据训练模型，模型再从真实状态附近生成短 imagined rollouts。

课程给出的完整 pipeline 是：

1. 用当前策略收集真实数据，加入 D_{env} ；
2. 用 D_{env} 更新动力学模型 $p_\theta(s'|s, a)$ ；
3. 从 D_{env} 的状态出发，在模型里用当前策略采样短 synthetic rollouts，加入 D_{model} ；
4. 用 D_{model} 更新策略和 critic，必要时也可混入 D_{env} 。

这里最重要的工程经验是“short rollouts from many real states”。讲者在口头解释里强调过，从真实轨迹末尾往后接长 rollout 并不稳妥；更好的做法是从 replay buffer 里很多中间状态出发，生成少量、短程、局部可信的 imagined transitions。

4.2 什么时候 model-based 值得做

课程没有把 model-based RL 神化，而是很克制地列出了利弊。优点包括：

- 如果模型好学，样本效率会显著提高；

- 模型可以在没有 reward 标注的情形下，用自监督方式训练；
- 模型有时是 task-agnostic 的，可以跨任务复用。

缺点同样扎实：

- 模型并不直接为任务回报优化；
- 有些领域里学模型比学策略还难；
- 额外引入模型，意味着额外超参数、额外算力、额外调试面。

课程给出的判断标准

是否使用 model-based RL，不该由“它看起来更聪明”决定，而应由“这个领域中的动力学是否足够可学”决定。模型越难学，planning 与 imagined rollouts 的收益就越不稳定。

4.3 本章小结

model-based policy optimization 可以看成 planning 与 model-free RL 的折中：不在测试时频繁搜索，而是在训练时用模型补充局部覆盖。它特别适合真实交互成本高、但局部动力学又相对可学的场景。

5 案例：灵巧手操控为什么适合做 model-based RL

5.1 任务定义与为什么它是好案例

讲者最后用 dexterous manipulation 作为案例，原因有两个。第一，五指灵巧手是高维、易碎、真实交互代价高的系统，样本效率特别关键；第二，这个任务能清楚展示 model-based 方法的关键部件，比如 ensemble、CEM planner、reward design 与真实系统的闭环执行。

Case study: Model-based RL for dexterous manipulation

Deep Dynamics Models for Learning Dexterous Manipulation

Anusha Nagabandi, Kurt Konoglie, Sergey Levine, Vikash Kumar
Google Brain

September 2019

Why this paper?

Still one of the most impressive results with five-fingered hands!

Nice comparisons on what is important for good performance

32

图 8: dexterous manipulation 被当作 case study 的原因：高维、真实交互昂贵，但又迫切需要高样本效率。

在这个案例里，状态包括手和物体的位置；动作空间是控制 24 自由度左右的五指手；奖励目标是跟踪指定物体轨迹，并惩罚掉落。这种任务如果完全依赖 model-free RL，真实世界数据成本会很高。

5.2 为什么 ensemble 和改造版 CEM 有帮助

课程提到，案例中使用的模型并不是单个网络，而是一个 ensemble。原因在于，不同模型会在不同区域犯不同错误，集成后能减轻单一模型过度乐观的问题。规划器方面，则采用 CEM 的变体，并加上 smoother action sequence 等技巧，以便搜索更平滑、可执行的动作序列。

Case study: Model-based RL for dexterous manipulation

Simulated experiments

Model-free methods:

SAC: actor-critic method

NPG: policy gradient method

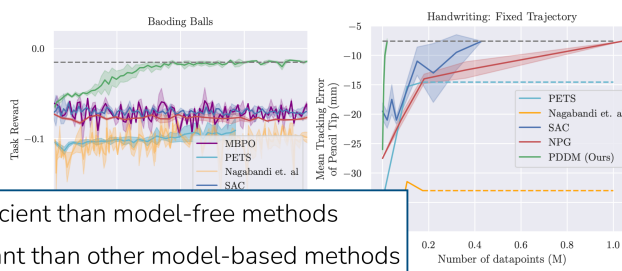
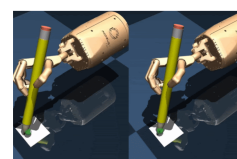
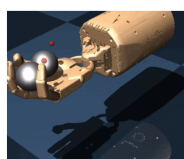
Model-based methods:

PDDM: proposed method

MBPO: RL with model-generated data

PETS: CEM-based planner

Nagabandi et al.: random shooting, no ensembles



More efficient than model-free methods
More performant than other model-based methods

图 9: 仿真实验结果表明：在该任务上，合适的 model-based 方法既比 model-free 更省样本，也比较弱的 model-based baseline 更强。

这恰好对应本讲前半段的理论：如果模型误差会误导规划，那 ensemble 就是一种减轻误差 exploitation 的办法；如果规划动作过于抖动，那就要在 planner 内加入更贴近物理执行的约束。

5.3 真实世界部署说明了什么

课程最后展示了 ShadowHand 上的真实系统结果：Bouncing balls 旋转等任务可以在有限小时级别的数据内学成。这说明 model-based RL 在硬件脆弱、交互昂贵的场景里尤其有吸引力，因为它能把有限真实经验“放大”成更多有价值的决策信号。

Case study: Model-based RL for dexterous manipulation

Real-world dexterous control with ShadowHand



- Efficiency is key for fragile hardware
- Learns Baoding ball rotation in ~4 hours
- Ball is reset with another robot arm

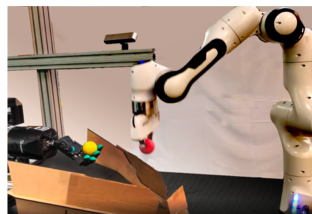


图 10: 真实世界 ShadowHand 实验凸显了 model-based RL 的核心价值: 在昂贵硬件上用更少真实交互获得可用策略。

5.4 本章小结

案例部分把整节课的抽象讨论落到了具体系统上。高维硬件控制之所以适合 model-based RL, 是因为真实数据太贵, 而局部动力学又并非完全不可学。此时, ensemble model、CEM planner、MPC 式闭环执行就会组合成一套可落地的工程方案。

6 总结与延伸

Lecture 11 的关键贡献, 是把“学模型有什么用”这个问题拆成了三层。第一层是世界模型本身: 你能否从数据里学到足够可信的动力学近似。第二层是决策调用: 你是直接在模型里做 planning, 还是把模型当成 imagined data generator。第三层是误差管理: 你怎样处理数据分布失配、open-loop 脆弱性、长 horizon 误差累积, 以及规划器对模型漏洞的利用。

从课程视角看, planning 与 model-free RL 并不是对立面。planning 适合短 horizon、需要强在线反应的场景; model-based policy optimization 适合用 imagined rollouts 提高样本效率; MPC 则是让 planning 进入闭环控制的关键桥梁。所有这些方法的共同前提都是: 模型至少在某个局部区域里足够有用。

如果把这节课和现代 RL、机器人系统联系起来, 可以得到一个很务实的结论: model-based RL 不是默认更高级, 而是默认更脆弱。只有在动力学确实可学、硬件交互成本确实高、而你又有足够工程能力管理模型误差时, 它才会成为真正有优势的路线。这也是为什么课程最后会落在 dexterous manipulation 这样的案例上, 因为它恰好满足了这些条件。